



Using a BL-HS motor driver chip and field-oriented control (FOC), the RoboMaster G300 Brushless DC Motor Speed Controller enables precise control over motor torque.

Exclusively designed for the RoboMaster M3000 PMS Brushless DC Gear Motor and G300 Brushless DC Motor Speed Controller, the U3000 Assembly Kit includes several cables and a terminal board.

RoboMaster System Specification Manual, RoboMaster System User Manual, Introduction of RoboMaster System Models

The M3000 Assembly Kit includes several cables and a terminal board, creating a complete assembly system driven by four independent motors.

ROBOMASTER

机甲大师超级对抗赛

技术方案

目录

1. 概述	1
1.1 背景&目标	1
1.1.1 导航部分	1
1.2 其它学校机器人分析综述	1
1.2.1 导航部分	1
1.3 机器人功能定义	4
1.4 机器人核心参数	5
2. 设计方案	6
2.1 算法设计	6
2.1.1 导航部分	6
2.2 其它	26
2.2.1 Git-Flow 工作流+Commit Message 规范	26
2.2.2 持续集成自动构建 Docker 镜像	27
2.2.3 开发文档记录	27
3. 研发迭代过程	29
3.1 测试记录	29
3.2 版本迭代过程记录	30
3.3 重点问题解决记录	31
4. 团队成员贡献	33
5. 参考文献	34

1. 概述

1.1 背景&目标

1.1.1 导航部分

由于哨兵机器人无操作手的特性，机器人的建图、定位、感知、导航、自主决策是使其在赛场上发挥作用的技术支撑。

建图部分，考虑到对抗赛的场地面积较大，建图算法需要具备一定的全局一致性。

定位部分，考虑到赛场有崎岖路段、坡面，而且机器人携带的传感器可能处于高速的运动状态，定位模块必须做到能在高速运动的情景下实现鲁棒的高精度定位，为导航提供信息。

感知部分指的是机器人对于障碍物、坡面等环境的识别，赛场地图中有上下坡、台阶等路段、赛场中有高速移动的其他机器人，哨兵需要依靠感知模块识别这些物体，并在导航时做出相应的调整，如：调整底盘角度防止卡在大角度坡面、绕行动态障碍物等。

导航部分在定位和感知的基础上进行，规划算法部分应尽量找到一条由起点到终点的最短路，控制算法部分应实现对路径的准确追踪同时保证机器人的平稳运行。

自主决策部分则须实现集合赛场信息，发布命令。一个逻辑清晰、能够可视化运行效果的决策模块将为哨兵与操作手的配合和调试带来极大的便利。

1.2 其它学校机器人分析综述

1.2.1 导航部分

方案	传感器选型与装配	算力平台	建图	定位	感知	规划	控制	决策
西电 2023 ^[1]	Livox MID360 装于云台、RealSense-D435 深度相机补充盲区			Point-LIO /Faster-LIO2		A *	cmu Autonomous Exploration Development Environment local-planner	行为树
华农 2022 ^[2]	思岚 A2 激光雷达、CH110 九轴陀螺仪	NUC10 I7		EKF+AMCL		A *		状态

								机
华农 2023 ^[3]	思岚 A2 激光雷达、 CH110 九轴陀螺仪	NUC10 I7		ICP+EKF		A *	PID Position Pursuit 轨迹 追踪算法 + 人工势场避 障法	行 为 树
太科 2023 ^[4]	Livox MID360 装于底 盘		Fast-LI O2	Fast-LIO2+a mcl		A *		
哈工业 2023 ^[5]	OUSTER OS0 装于云 台		Fast-LI O2	IEKF		A *	DWA	状 态 机
沈阳航空 航天大学 2023 ^[6]	Livox MID360 装于底 盘		Fast-LI O2	Fast-LIO2+I CP/DLL+imu filter		Di jks tra	TEB	行 为 树
中南大学 2023 ^[7]			POINT- LIO/Fas t-LIO2	ICP(初始) +POINT-LIO /Fast-LIO2	linefit_gro und_segm entation	Di jks tra	TEB	
华中科技 大学 2023 ^[8]	Livox MID360 装于云 台	NUC11 I7 RTX20 60		DLO	点云映射 点云滤波		TEB	
百度 Appolo	16 线/128 线 velodyne			MSF	基于 CNN 的 障碍物预 测+障碍 物聚类+ 后处理+ 障碍物预 测			

传感器方面，综合分析往年的哨兵实现方案和近期快速发展的自动驾驶技术，不难看出，

3 维激光雷达逐渐替代 2 维激光雷达成为移动机器人实现定位和建图功能的主要传感器。而在算力平台的选择上, NUC 以其轻巧和高性能的特点成为了几乎所有学校的选择。

在建图层面, 随着机器人技术的发展, LIO-SLAM 以其较高的精度和较低的成本, 逐渐成为了工业机器人的 SLAM 解决方案。常见的有 Fast-LIO^[9,10]、Point-LIO^[11]、LIO-SAM^[12]等, 建立高精度的 3 维点云地图已经不再是一个难题。Fast-LIO 提出 ikd-Tree 数据结构, 实现高效的激光雷达直接原始点云配准方法, 但其缺点在于缺乏后端, Z 轴漂移严重, 大场景下容易出现回环无法闭合的情况; Point-LIO 采用逐点更新的框架, 能够对极端剧烈的运动进行状态估计, 问题是该框架对参数敏感, 建图精度较低; LIO-SAM 提出基于因子图的框架, 适合多源传感器融合, 其优化及回环检测的功能使其在建图上取得了较好的全局一致性, 缺陷在于对于 IMU 的要求较高。

在定位方面, 较为常见的是紧耦合的 LIO, 这些工作主要分为迭代扩展卡尔曼滤波和滑动窗优化两种状态估计框架, 充分考虑了 Lidar 观测和 IMU 观测的内在约束性, 互相影响, 共同决定最终的结果。

由于自主导航的需求, 还有一个需要解决的问题是重定位, 也就是如何确定机器人在导航地图中的位置。各个学校常用的方法有 AMCL^[13]、基于 ICP 类^[14,15]点云配准算法的重定位方法和 DLL^[16]等。AMCL 采用粒子滤波器来跟踪已经知道的地图中机器人位姿, 对于大范围的局部定位问题工作良好, 但是它对地图构建的算法要求高, 更容易受到环境的影响, 在高速移动的机器人上效果不佳。ICP、NDT 之类的点云配准算法通过迭代的方式寻找两个点云之间的最佳变换, 以较大的计算代价换取了较高的精度。DLL 提出了一种快速直接的基于地图的激光雷达定位方法, 与 ICP 和 NDT 相比, 该方法速度快一个数量级, 精度与它们相当, 同时比蒙特卡洛定位方法鲁棒, 但在高速旋转的情景下, 该算法同样容易失效。

此外值得一提的是百度 Apollo 的 MultiSensorFusion^[17]定位方法, 直接在 pre-build map 中实现定位, 同时解决了定位和重定位的问题。它计算当前点云与先验地图的误差, 作为 Fusion 的一个因素, 最后融合 IMU 和 GNSS 消息获得定位。

在感知方面, Navigation2 框架^[18]中的简单方法是对输入点云进行高度判断, 高于一定阈值的点云数据则被认定为是障碍物, 但这样简单的判断方式会将坡面等误判为障碍物。青工会分享中, 有学校提到的障碍物辨识方法是将当前的雷达扫描与预先建好的点云地图进行相减, 不为 0 的部分则被认定为障碍物, 不失为一个简单有效的解决方案。还有中南大学使用

的 `linefit_ground_segmentation`^[19,20]方法，在快速辨识和分割障碍物及地面上表现出色，对于坡面的效果也很好。

在规划控制方面，Navigation2，Movebase 框架为导航提供了工程化、成熟的解决方案，最常用的全局规划算法是 A*，局部控制算法是 TEB、DWA，但是由于赛场机器人小陀螺等其它功能的需求，这些现成的算法还需要进一步修改。不同于常用的基于 2d 栅格地图的导航算法，基于可视图的 FAR-Planner 算法^[21]在全局路径规划上也能取得不错的效果，但其主要问题在于地图精度和规划性能难以权衡，工程化程度不高，难以修改地图。此外西电选用的局部控制器是 CMU Autonomous Exploration Development Environment^[21]中的 local planner，该控制方法通过检查预先生成的路径与障碍物的碰撞情况排除不可行的路径，然后选择到达控制目标点最短的路径执行。该方法简单快速，但是由于预先生成的路径库较小，且算法本身基于阿克曼运动学设计，容易出现路径跳变的问题且在全向机器人上表现不佳。

决策方面，状态机和行为树是比较常见的选择。

1.3 机器人功能定义

- 哨兵功能
 - 建图功能
 - 定位功能
 - 感知功能
 - 识别坡面、阶梯
 - 规划控制功能
 - 动态避障功能
 - 自主决策功能
 - 收发裁判系统消息
 - 综合自瞄和感知模块的信息
- 功能量化
 - 建图精度高
 - 实现回环闭合
 - 定位鲁棒、精度高
 - 在小陀螺、通过崎岖路面、撞击等情况下不失效
 - 与实际距离偏差在厘米的量级

- 感知实时准确
 - 正确识别动态障碍物并在代价地图中以 20hz 的频率更新
 - 正确识别坡面，计算坡面法线与实际的偏差不超过 10%
 - 正确识别阶梯，正确识别率 90%以上
- 控制平滑且跟踪性好

1.4 机器人核心参数

名称	参数
重量、重心	25kg
尺寸（长宽高）	680*680*700
主要传感器型号、参数、数量	导航算力平台：NUC11; 激光雷达: Livox MID360+内置 IMU
机器人其它核心性能参数	导航平地平均移动速度 0.8m/s，最大速度 1.5m/s

2. 设计方案

2.1 算法设计

2.1.1 导航部分

2.1.1.1 功能简介与 pipeline

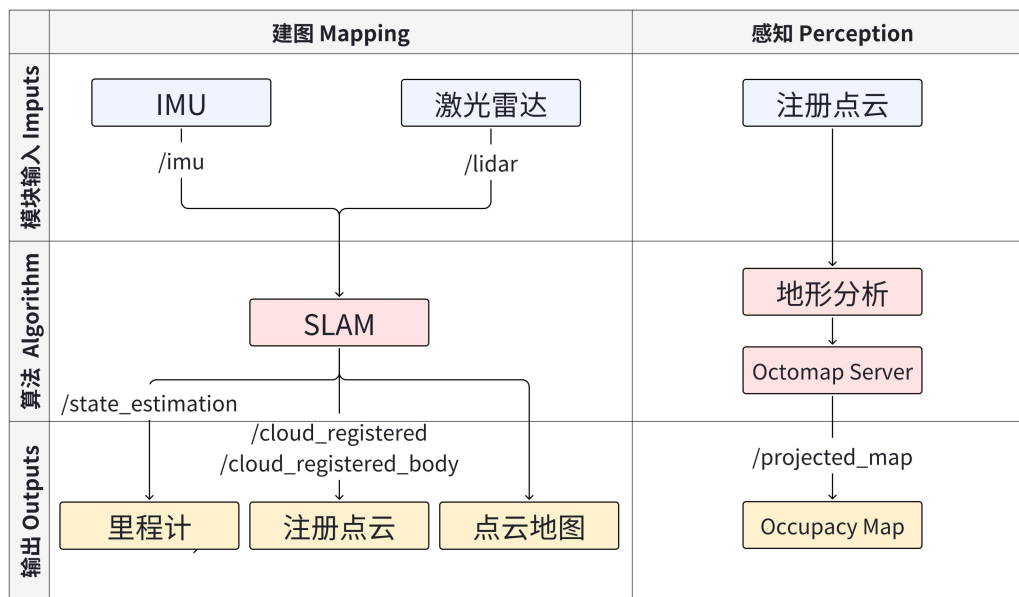


图 1 建图模式框架

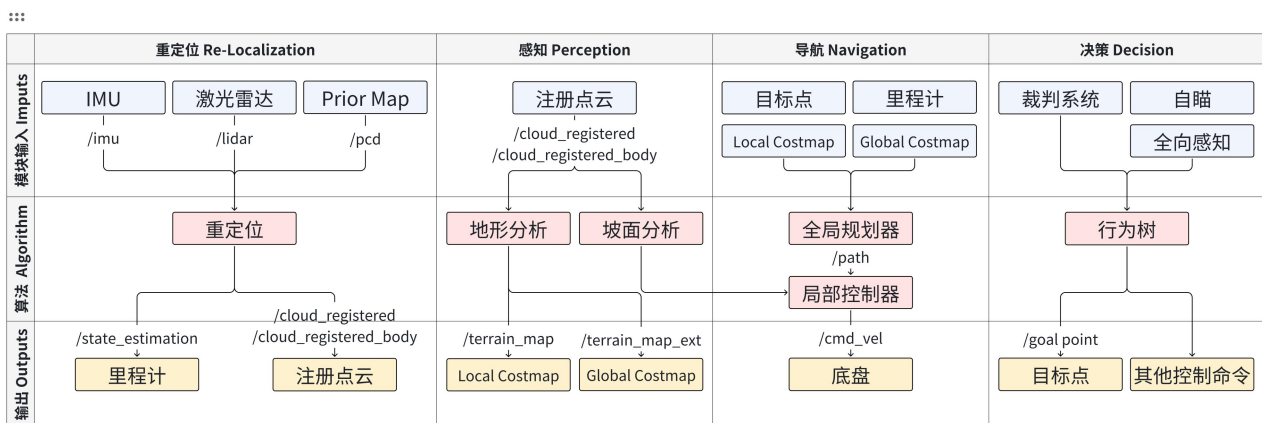


图 2 比赛模式框架

哨兵导航分为两个运行模式，一为建图模式：运行 SLAM 算法获得赛场的 3D 点云地图，为后续的重定位提供 Prior Map。同时感知模块识别分割出障碍物，使用八叉树将障碍物点云

进行下采样，然后将障碍物投影至 2D 的占用地图，为后续导航提供导航地图。由于两个地图是同时构建的，他们具有相同的原点。

二为比赛模式：重定位模块实现机器人在 Prior Map 和导航地图中的定位；感知模块识别坡面和动态障碍物，为规划和控制提供实时信息；导航模块中的全局规划器基于 2D 占用地图寻找由起点到达终点的可行路径，局部规划器追踪规划出的路径，转化为对底盘的控制命令，实现机器人的移动；决策模块收集裁判系统、自瞄以及全向感知的信息，综合做出判断指导哨兵机器人的行为。

2.1.1.2 重要算法原理阐述

建图：Fast-LIO-SAM^[22]

考虑到 RMUC 的赛场环境较大，建图算法需要具备较高的精度和全局一致性。Fast-LIO2^[10] 为紧耦合的 LIO SLAM 系统，其利用 ikd-Tree 的数据结构，实现了高效的激光雷达直接原始点云配准，但其缺点在于缺乏后端，Z 轴漂移严重，大场景下缺少全局一致性，会出现无法闭环的问题。Point-LIO^[11] 采用逐点更新的框架，虽然在极端剧烈运动的情境下展现出极高的鲁棒性，但也由于其点到面的配准方式约束不足的原因在建图精度上略逊一筹。LIO-SAM^[12] 基于因子图进行优化，取得了较好的全局一致性，缺陷在于对于 IMU 的数据非常敏感。

综合各个框架的优缺点，我们最终选择使用 Fast-LIO-SAM 作为我们的建图算法。它将 LIO-SAM 的后端 GTSAM 优化部分移植到 Fast-LIO2 的代码中，利用了 Fast-LIO2 优秀的前端框架，改进了其缺乏全局一致性的缺陷。

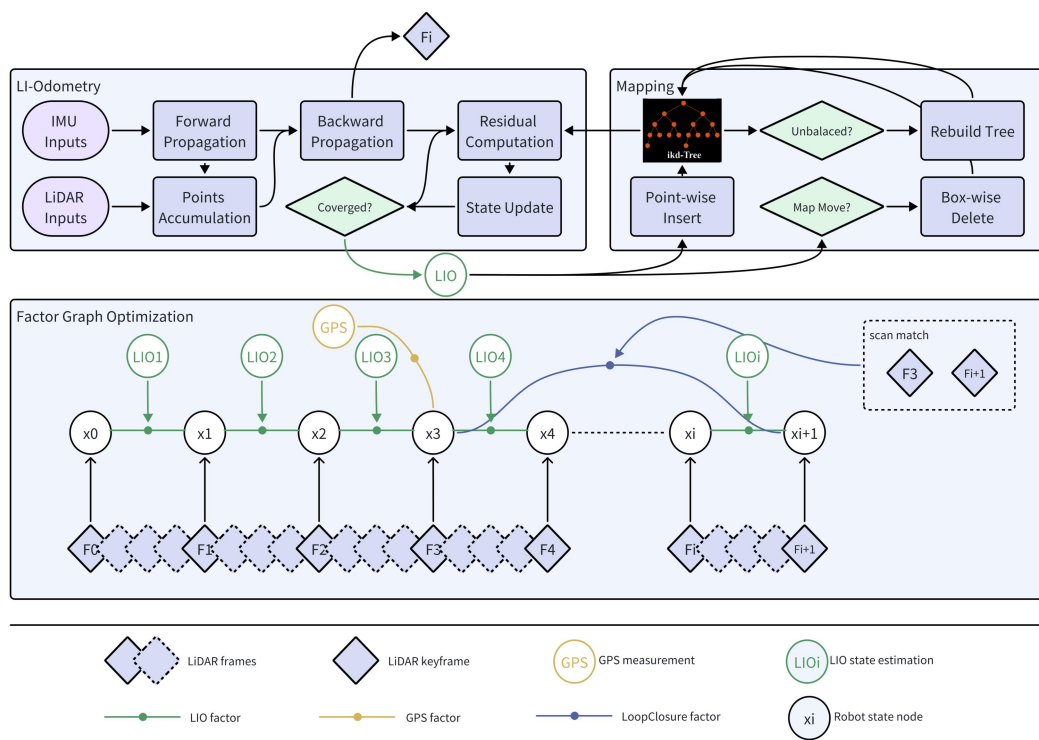


图 3 Fast-LIO-SAM 算法流程图

该算法将 Fast-LIO2 估计出的机器人位姿作为优化的一个因子，并在运行过程中保存关键帧。前端线程运行的同时，回环线程对关键帧进行 ICP 匹配，当检测到回环，则添加回环因子。如有 GPS 测量数据，则在数据到达时加入 GPS 因子。

在以上 2 或 3 个因子的约束下，使用 GTSAM 算法库进行基于因子图的优化，对轨迹以及地图进行纠偏和重建，使得全局一致性和建图的精度得到提升。

定位：Fast-LIO in prior map

RoboMaster 的赛场环境对定位算法提出了以下要求：1.地图是预先可知的；2.机器人在高速运动；3.需要厘米级别的精度；4.至少 20hz 的更新频率；5.算力有一定限制。

普遍使用的方案是使用里程计进行状态估计，维护机器人由起始点到当前位置的信息；再加上重定位模块，维护机器人到全局地图的位置信息；两者结合获得机器人在全局地图中的位置。

里程计部分，Fast-LIO2 凭借其快速及高精度的优势得到了普遍的使用。重定位部分，AMCL 对于大范围的局部定位问题工作良好，但是它在高速移动的机器人上效果不佳，不适用于赛场环境。ICP^[14]、NDT^[15]之类的点云配准算法算力消耗过大，容易使得电脑卡顿导致

导航模块失效，且 LIO 算法和 ICP 算法同时都在解决点云配准和 IMU 状态估计问题，比较冗余。DLL^[16]与 ICP 和 NDT 相比更快且精度相当，但在高速旋转时该算法容易失效。百度 Apollo 使用的 MultiSensorFusion 定位方法^[17]，计算当前点云与先验地图的误差作为 Fusion 的一个因素，最后融合 IMU 和 GNSS 消息获得定位，为我们方案的提出提供了灵感。

分析赛场需求和各个框架的我们提出对 Fast-LIO 进行修改，实现了 relocalization 运行模式，在一个模块内实现定位和重定位的功能。

Mapping 模式（即原本的 Fast-LIO），需要将当前 scan 与增量更新的 ikd-tree 中的点云进行配准，作为 ekf 迭代更新的一个观测。而在 relocalization 模式下，运行步骤有所不同：1. 得到第一帧点云输入后，使用 ICP 进行 scan-to-map 匹配（需要提供大概的初始位姿），得到当前机器人在 prior map 中的位置；2.对 prior map 进行下采样和移动，使得 prior map 的原点与当前机器人的起始位姿对齐；3.将 prior map 载入 kd-tree 中；4.关闭地图增量更新，使得配准在预先建好的地图载入的 kd-tree 中进行。其他部分则与 mapping 模式相同。

Algorithm 1: Fast-LIO in prior map

Input: Lidar raw points in current scan;

Imu input during current scan;

Prior map;

Initial pose.

Output: State estimation.

1 IMU initialization;

2 Backward propagation to compensate motion;

3 Register current scan to prior map with initial pose by ICP;

4 Downsample and transform prior map to align map origin with odom;

5 Load downsampled prior map into kd-tree, disable incremental map update;

6 Same as mapping mode in Fast-LIO;

此算法简单实用，实现方法精简，而且避免了地图增量式更新在长时间运行的情况下对内存带来的负担。

Fast-LIO2 的算法原理在其论文中已经说明详尽，此处不再赘述。

感知

Terrain_analysis^[21]

环境感知指的是机器人对于周围障碍物、地形的辨识能力。简单的解决方法如 navigation2 中 costmap 的 obstacle layer，将输入点云中高于一定高度阈值的点分类为障碍物。不难看出，这样的方法不仅会使坡面成为不可通行的区域，对于不同的障碍物分类性能也较差。

为了在建图模式下正确辨识坡面，在比赛模式下辨识动态障碍物，我们引入了地形可通

过性分析模块为机器人赋予环境感知的能力。

算法流程如下：1.接收和预处理环境点云。为了加快处理速度，仅保留车体附近的点云，并进行下采样，然后点云被存入体素网格中以体素网格为单位进行分析处理；2.计算每一个体素格的高程信息。对某一个体素网格中的所有点按照高度进行排序，按照设定的阈值将排在某一位置的点云的高度作为该体素网格的地形高度。3.计算该体素格中每一个点的高程信息。将每一个点的高度与该体素格的地形高度相减，得到的既是该点的高程信息。

在代码实现上，点的高程信息被存储在点云消息的 *intensity* 域，通过将 *intensity* 高于一定阈值的点辨识为障碍物即可实现障碍物和的辨识。

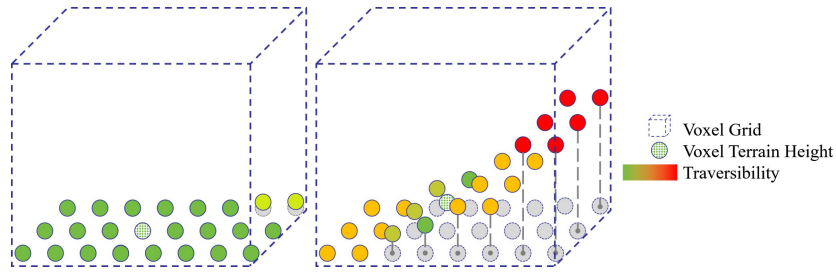


图 4 地形可通过性分析示意图

pathNorm 路径点云分析算法

对于需要上坡的情景，我们在测试的过程中发现了以下两个问题：1.全向轮机器人如果不以特定的角度对齐坡面，则会发生上坡过程中两个轮子悬空的情况，导致机器人卡住；2.使用的导航算法是基于 2D 地图的，对于上坡的情景会使用同样的速度，导致机器人上坡时速度很慢。

为了解决以上两个问题，我们提出了路径点云分析算法，该算法接收规划算法输出的路径点和点云，分析路径上点云的坡度，在坡度超出一定阈值时向控制器发出信号，控制机器人进行底盘-坡面对齐和上坡加速。

算法流程如下：接收导航规划的路径点集合 $Path$ ，其中包含 m 个 $PathPoint$ 。对输入路径进行处理，每机器人长度 L ，保留一个路径点，得到路径点集合 $Path'$ ，其中包含 n 个 $PathPoint$ 。每相邻的两个 $PathPoint \in Path'$ ，为一个路径点对 $PathPointPair$ 。遍历所有路径点对，在两个路径点之间裁剪出以路径点对间距（即 L ）为长，机器人宽度 W 为宽的矩形点云块 $PointCloudSquare$ 。对 $PointCloudSquare$ 进行半径滤波处理，滤除噪声。计算 $PointCloudSquare$ 内每个点的法线^[23]，最后得到每

一个 *PointCloudSquare* 平均法线和平均高度作为当前 *PointCloudSquare* 的坡度信息和高度信息。

Algorithm 2: Path Normal Analysis

Input: Path;
Surrounding pointcloud.
Output: PathNormals and PathNormals

```

1 Path'=Keep PathPoint per L;
2 PathPointPair={pointi, pointi+1}, pointi ∈
  Path' for each PathPointPair do
3   Filter out PointCloudSquare using crop hull;
4   Apply radius outlier removal;
5   Compute normal of each point in PointCloudSquare via least square
    method;
6   Compute average normal and average height of PointCloudSquare;
7   Add normal and height of this PointCloudSquare to the end of
    PathNormals and PathHeights;
8 return PathNormals and PathNormals.
  
```

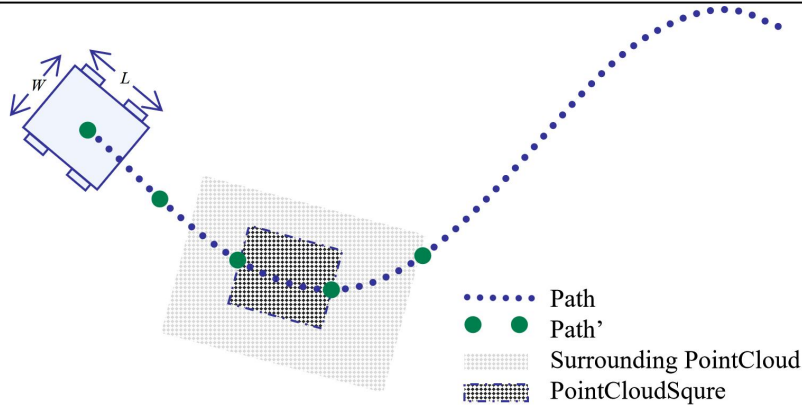


图 5 路径点云裁剪示意图

坡面辨识

坡面辨识的实现相对简单，若点云块法线与 z 轴的夹角大于一定的阈值，且点云块的高度高于机器人所在位置，则认为下一步机器人将要上坡。那么，路径分析模块将对控制器发出控制信息，将小陀螺停止并将车体旋转到适合上坡的角度，并依据坡面的坡度进行不同比例的加速。

如果点云块法线与 z 轴的夹角大于一定的阈值，但点云块的高度低于机器人所在位置，则认为机器人在下坡，不进行底盘对齐和加速操作。

台阶辨识以及单向通行路段

台阶的辨识是在以上裁剪出路径点云块的基础上实现的。对于每一个 *PointCloudSquare*，统计其中的点云数量，对其中的点云进行高度排序，得到最高的高

度和最低的高度；设裁剪阈值,裁剪高于最低点且低于最高点阈值高度的点,并统计剩余点云数量；设比例阈值,如果剩余的点云数量高于比例阈值,则此 $PointCloudSquare$ 对应的地形为一个台阶,否则为一个坡面。

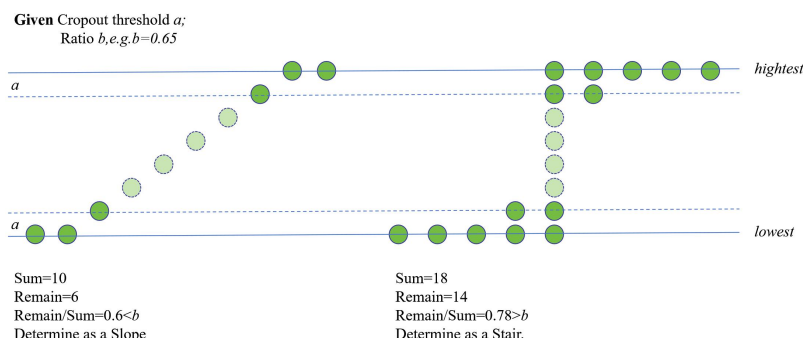


图 6 台阶辨识示意图

此外还需要做的是对导航地图的处理,下台阶是单向的,也就是说台阶处可以下行不能上行。此功能通过在 navigation2 中自定义动态 static layer 实现,当机器人在台阶的高侧,则可通行,在低侧则动态 static layer 出现,阻挡上行路径。

下台阶的初衷是为了缩短哨兵由 R1 高低前往崎岖路段的路径,但由于机械悬挂的限制和稳定性的要求,此功能最终没有被应用。

规划控制: Theta* Planner + MPPI Controller + Constrained Smoother

Theta* Planner

A*作为一个基于网格的规划器,其搜索方向为8路网格联通方向,因此搜索网格被限制在 45° 或 90° 航向上。而Theta*即要打破这种固定角度航向的弊端以此来得到更优和更短路径,主要是通过Line-of-sight algorithm^[24]来判断是否优化路以及直连。

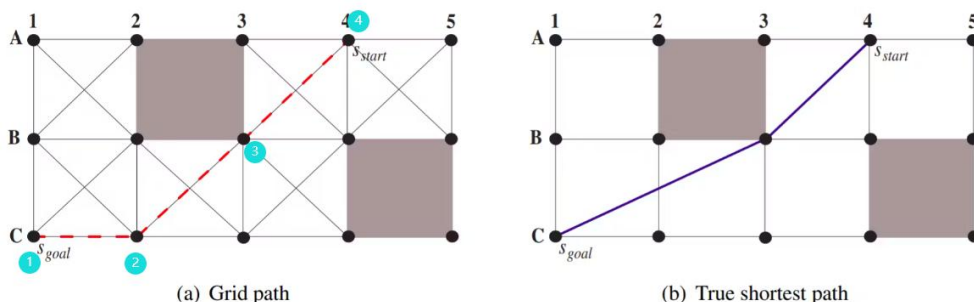


图 7 GridPath 的弊端。如左图 A*进行搜索时路径为 4-3-2-1,但实际上可直接从 3-2-1 替换为 3-1 才是最短路径。而这个是否替换便是 Line-of-sight algorithm,检查到 3-1 之间存在视线所以就直接去除 2 以此来完成最短路径搜索

此外值得一提的是哈尔滨工业大学 RM2023 赛季开源的哨兵技术方案中也提到了 A* 算法直接使用的弊端，那便是路径过长以及曲折弯绕，他们采用了 A* 后平滑技术来减少路径点数和直连，但是这种方法在一些细节上可能工作不佳，例如下图所示，核心在于 A* 后优化路径依旧是基于 A* 搜索出来的初始路径，而无法纠正 A* 此前犯下的方向选择错误。

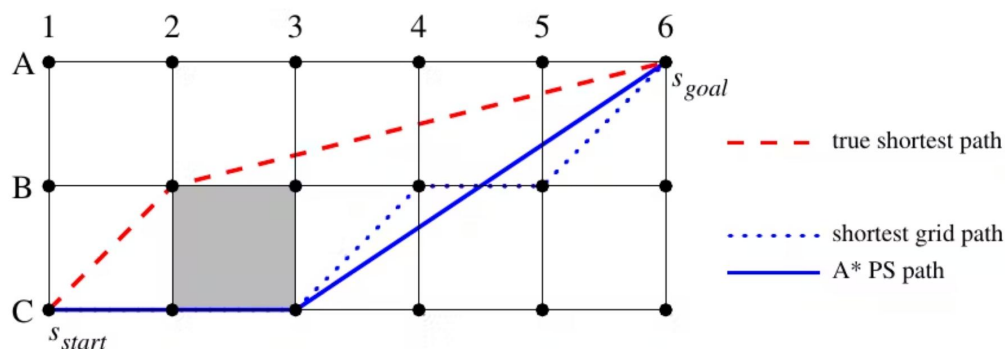


图 8 AstarPS problem

下图对比了三者在 costmap 环境下的表现：

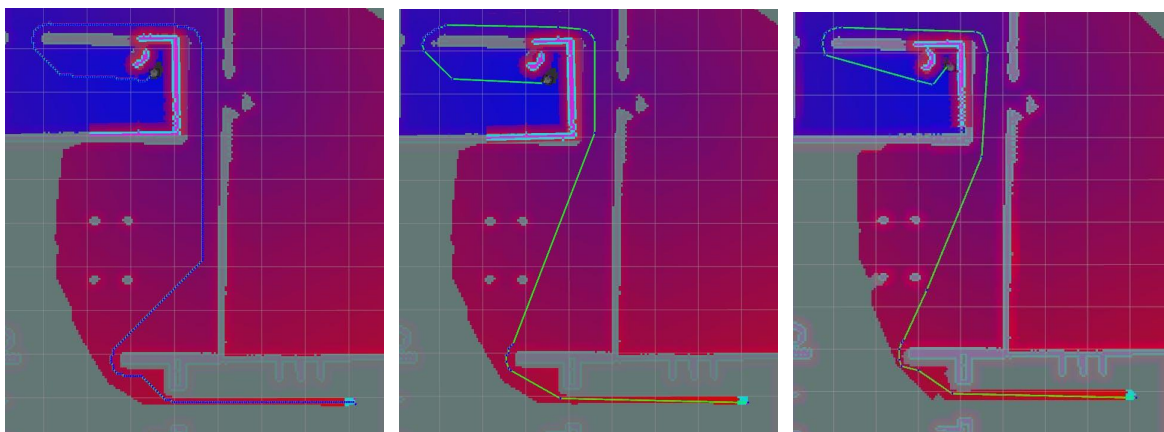


图 9 A*: 22.25m A*-post smooth: 21.76m Theta*: 21.53m[25]

而 Theta* 算法也有其弊端，也就是在于 LOS 检查带来的消耗，Theta* 在每次扩展到一个新的顶点 s 时，都会将所有 s 的可见邻接点都与 $\text{parent}(s)$ 计算一次 LOS，然后更新这些邻接点的 g 值和父节点，再将它们添加到 open list 中去，这导致每扩展一个点都要进行大量的 LOS 计算，而实际上并不是所有邻接点的 LOS 计算都是必要的，因为很多点最终并不会被访问到。因此引入 Lazy Theta* 以减少 LOS 的次数以此提升效率，Lazy Theta* 在扩展到一个新的顶点时，并不对邻接点的 LOS 进行计算，而是乐观地认为所有可见邻接点与 $\text{parent}(s)$ 都存在 LOS，于是直接修改它们的 g 值和父节点，然后添加到 open list 中去。当从 open list 中 pop 出某个顶点，需要真正以这个顶点为基础开始扩展的时候，再去验证它与父节点之间是

否真的存在 LOS，如果存在，则什么也不用改变，如果不存在，则重新计算它的 g 值和父节点。这样，只有那些真正被探索到的点才会实际执行 LOS 检查，大大减少了 LOS 的次数。而 Navigation2 采用的是 LazyTheta* 的进一步变体，LazyTheta*-P。

但此时 LazyTheta*-P 还是有一些小问题，那便是他太想走直线了，导致走弯道时是多直线拼接导致钝角衔接，对机器人的平滑控制造成了一定的影响，因此需要使用平滑器对路径进行优化处理。

Model Predictive Path Integral Control^[26]

比较常用的全向控制算法有 DWA 和 TEB。基于优化的 TEB 算法易于综合处理各类软硬约束，规划曲线连续性易于保证，不足在于优化变量越多，耗时越久，且易陷入局部最小值。基于空间采样的 DWA 算法模型简洁，实时性好，动态环境下避障能力强，但是其规划出的轨迹短且单一，前瞻性不足。

综合以上原因，我们最终选择了一种机动性高的控制器 MPPI 以适应赛场机器人的需求。

MPPI 的是基于随机采样和近似推理的方法。首先，它从当前状态开始，使用采样函数生成多个随机样本，并根据这些样本计算每个可能的轨迹的性能(代价函数)。然后，通过对这些性能值进行加权求和，并将其作为每一个控制策略的权重，选择具有较高权重的策略进行下一步操作。这种基于概率分布和随机样本生成的方法可以避免陷入局部最优解，并且适用于非线性和非高斯噪声环境。

作为机器人的局部控制器，MPPI 模拟多个随机化输入来预测系统行为，这使其获得了较好的前瞻；MPPI 模拟未来数千条轨迹，被阻塞的路径具有很高的代价，这使其获得了类似于空间采样类算法优秀避障能力。除此之外，此算法易于调试，对于路径追踪的各种需求，如是否避免转向、是否对齐路径方向等，可以通过简单增加 Critic，即修改代价函数和各个代价的权重来实现；其中还有与跟踪性和平滑性相关的 Critic，使得平滑的控制易于实现。

接下来讲述我们尝试过的一个导航框架，并将指出此框架最终没有被选用的原因。

FAR-Planner

赛季之初，我们分析总结了上赛季使用 Navigation2 的一些问题：1..路径规划效率不高，空间浪费（栅格的分辨率不依赖于环境的复杂度）；2.机器人容易卡在膨胀层中，且对应的恢复行为效率低下。

而CMU 的工作 Autonomous Exploration Development Environment 以及在此框架上开发的 FAR-Planner 从理论上正好可以解决以上两个问题：1.使用可视图对环境进行表示，用于导航算法搜索路径的节点仅在环境中的拐点中出现，环境表达高效；2.不是基于代价地图的导航，没有膨胀层等概念，机器人不会被算法卡住。

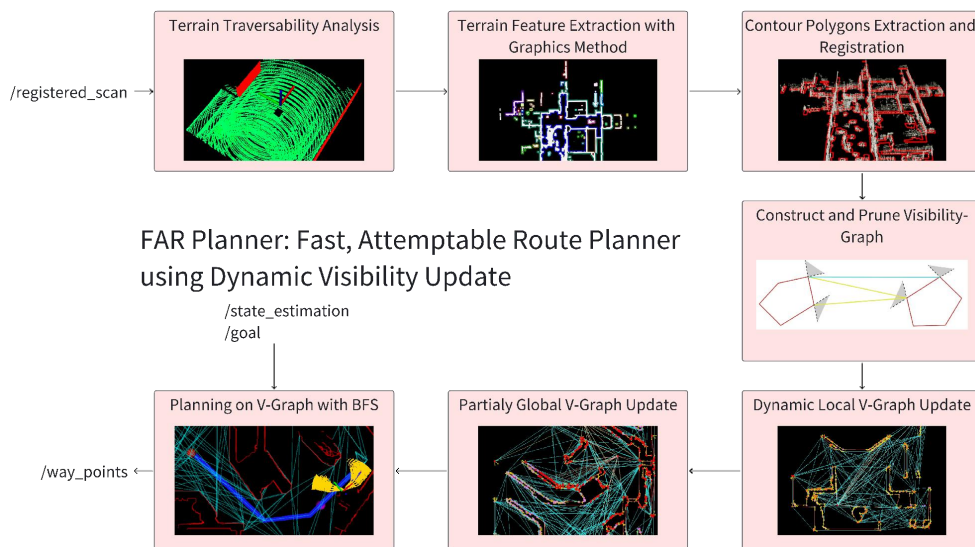


图 10 FAR-Planner 算法流程

FAR-Planner 首先使用图形学的方法对输入点云进行特征提取；然后从点云转化为的鸟瞰图图像中提取多边形，去除不必要的节点，彼此“可视”的节点之间的连线为一个可视边，所有可视边及节点构成可视图；在建图的过程中动态地、局部的更新可视图；最终，使用广度优先搜索的方法在地图中寻找当前机器人到达终点的最短路。

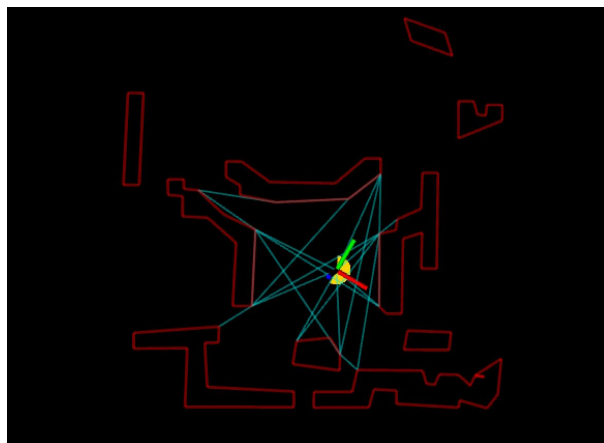


图 11 FAR-Planner 规划算法（红色多边形为点云提取的障碍物，蓝色连线为可视边）

虽然该算法提出了加速可视图构建的方法，使得基于可视图的实时规划成为可能，但此

框架还是有如下难以克服的缺点：

1. 工程化不足，地图难以编辑和修改；

2. 地图的精度和规划的性能难以权衡。前文提到，会对输入点云进行图形学处理，通过调试发现，由于我们使用的 Mid360 线数较少，在低矮障碍物上的点云也很少。相应的，如果为了保证低矮障碍物被正确识别，则需要提高地图的分辨率，但同时会大大增加边框角点的数量，使得搜索的时间大幅度增长。如果降低分辨率，不仅部分障碍物不能被正确识别（图 13 左），而且容易导致狭窄路段的两侧直接连通，路径无法通行（图 14 左）。

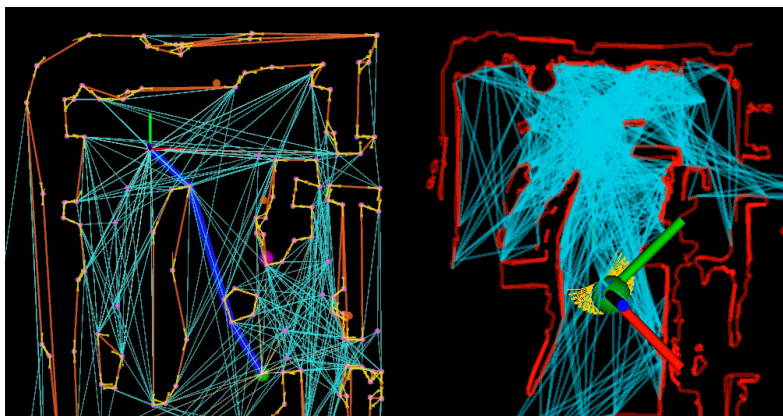


图 12 低分辨率导致部分障碍物无法识别（左）高分辨率导致可视边过多（右）



图 13 可视图地图（左）和栅格的地图（右）对比

虽然在赛场环境下，环境中低矮的障碍物比较少，这个算法仍有一定的适用性，但是考虑到 Navigation2 框架极高的工程化程度以及栅格地图在各种环境下的普适性，我们最终还是选用了基于栅格地图和代价地图的环境表示方法。

Local Planner

框架中的局部控制器，Local Planner 是一中基于采样的局部规划算法，其基于阿克曼

类小车设计。它考虑车体最大转向范围，在车体正前方 $\pm 16^\circ$ 的范围内使用三次样条插值绘制出一系列曲线存储在文件中，作为运动基元库供后续选择。运行过程中，Local Planner 通过检查路径基元库与障碍物点云的碰撞，去除不可通行的路径，并在剩余的可通行路径中，选取代价最小的路径对应的方向作为最终的执行输出。

此方法：1.将采样离线化，提升了运行的效率；2.路径都是平滑曲线，在阿克曼类小车上实现了平滑的控制；3.实现了有效的实时动态避障。

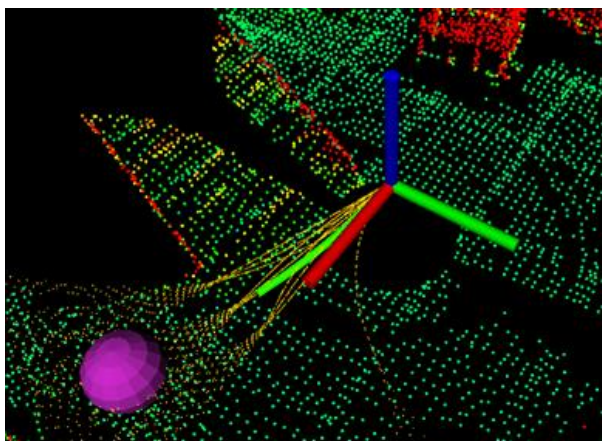


图 14 Local Planner(粉色点为局部目标点，黄色曲线为无碰撞运动基元，绿色曲线为最终选择的路径)

实测过程中，我们发现了以下几个问题：1.由于运动基元库中存在相交的曲线，在空旷的场景下，会出现几条路径都无碰撞可通行，代价得分相近，最终选出的路径在这几个运动基元之间频繁跳变，甚至导致执行的速度频繁正负变化，虽然最终可以到达目标点但对于电机的伤害较大。2.将全局规划器更改为 Navigation2 中的算法后，两者兼容性一般。虽然通过一定的处理可以实现，但是其性能也许并不值得这些时间成本。

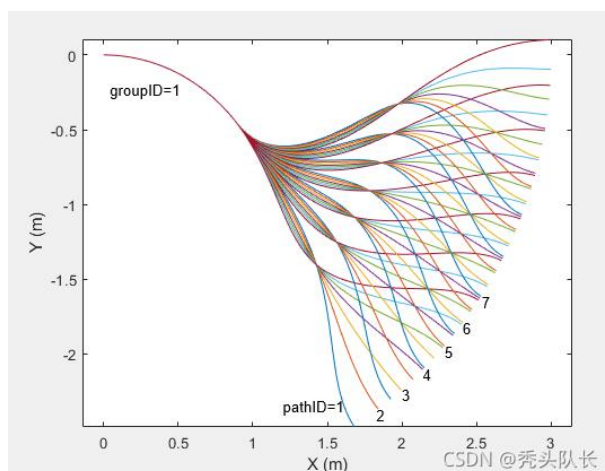


图 15 路径基元库^[27]

Constrained smoother

除了追求尽可能短的路径，全局路径的平滑程度对于控制器的跟踪也有很大的影响，对机器人运行的安全性也起到一定的作用。因此，在 Planner 之后加入 Smoother 很有必要。

Navigation2 框架提供了 3 中路径平滑器：Simple Smoother, Savitzky-Golay Smoother 以及 Constrained smoother。他们的特点、应用场景和原理如下：

1. Simple Smoother

特点是简单快速，但会忽视所有动力学的约束条件；其原理是对原始路径和平滑后的路径进行加权，以获得一条兼具原始路径特点又减少了扭曲和抖动的路径。其加权公式为：

$$\begin{aligned} x_i^{origin} &= path_{origin}.poses[i].x, & y_i^{origin} &= path_{origin}.poses[i].y, \\ y_{i-1}^{new} &= path_{new}.poses[i-1].y, & y_{i+1}^{new} &= path_{new}.poses[i+1].y, \\ y_i^{new} &= y_i^{origin} + W_{data} * (x_i^{origin} - y_i^{origin}) + W_{smooth} * (y_{i-1}^{new} + y_{i+1}^{new} - 2 * y_i^{origin}) \end{aligned}$$

2. Savitzky-Golay Smoother

是一种基于 Savitzky-Golay 滤波器原理的简单高效的平滑算法，适用于所有类型的规划器，但无法对较大的不连续或抖动路径惊醒处理。

3. Constrained smoother

将轨迹平滑构建为一个无约束的非线性最小二乘优化问题，用 Google 开源的非线性优化求解库 ceres-solver 对问题进行求解。

平滑项代价：

$$C_{smooth} = \frac{1}{2} \sum_{i=1}^{n-1} (\Delta x_{i+1} - \Delta x_i)^2$$

曲率项代价，当 kappa 满足最大曲率约束时此项为 0：

$$C_{curvature} = \frac{1}{2} \sum_{i=1}^{n-1} ((\kappa_i > \kappa_{max}) ? (\kappa_i - \kappa_{max})^2 : 0)$$

参考点偏移项代价，约束与原始离散点的偏移：

$$C_{distance} = \frac{1}{2} \sum_{i=1}^{n-1} \|\Delta x_i - \Delta x_{i_{origin}}\|^2$$

障碍物距离代价，通过压入 costmap 最小化 cost 来远离障碍物，自行车用多圆模型表示，每个圆心的 cost 代价可不同：

$$C_{cost} = \frac{1}{2} \sum_{i=0}^{n-1} \sum_{j=0}^{m-1} w_{ij} * cost_{ij}^2$$

最终的优化目标函数为非线性最小二乘问题，最小化所有残差块至收敛：

$$\min(w_{smooth}C_{smooth} + w_{curvature}C_{curvature} + w_{distance}C_{distance} + w_{cost}C_{cost})$$

但算力开销较大，需要控制算法的执行次数以节省开销。

决策

决策框架

在全自动机器人开发的过程中，我们的目标之一是寻求一个既简洁又具备扩展性的逻辑方案，以提升开发迭代的效率。在众多行为系统中，有限状态机（FSM）因其简洁性而备受推崇。然而，FSM 的局限性在于，状态之间的转换需要依赖特定的转换逻辑，这在一定程度上削弱了其模块化特性。

相比之下，行为树（BT）作为一种树状结构，其节点代表不同的行为，而非状态。在 BT 中，节点之间呈现出一种层次关系，每个节点的执行结果（成功、失败或运行中）均由其父节点管理，从而决定后续的行为。这种设计不仅增强了节点的模块化特性，还使得逻辑更加直观易懂，有助于开发者避免陷入复杂的连线逻辑中。

因此，行为树的主要优势在于其优秀的封装性和模块化设计，大幅度提高了开发者的工作效率。且行为树直观简洁，方便与其他机器人的操作手进行战术沟通。

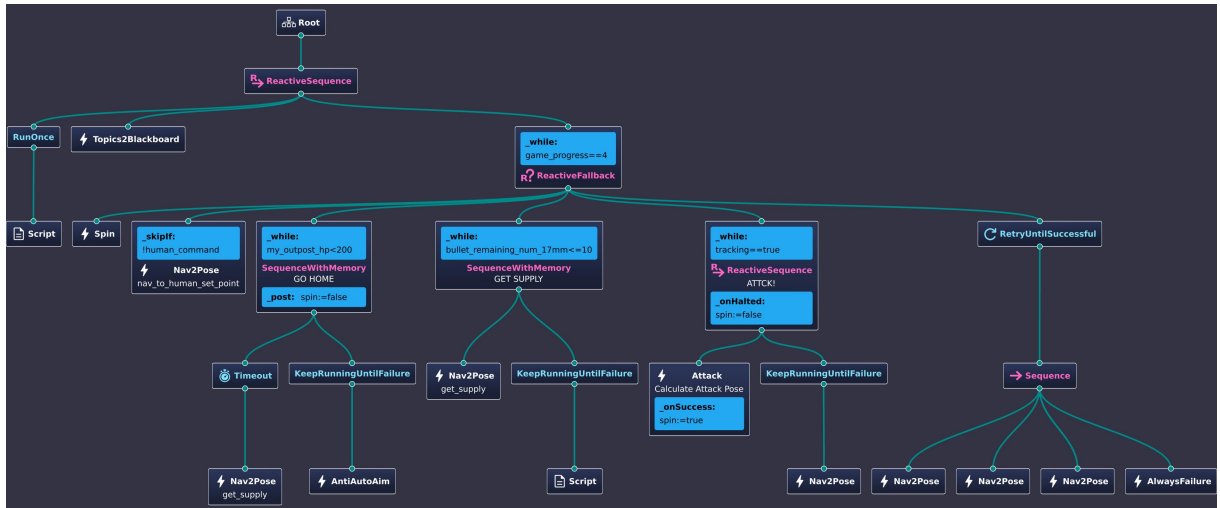


图 16 行为树决策框架

追击行为

当得到自瞄模块的敌人位置信息后，机器人会移动至距离敌人约 3m 的位置进行打击，如果敌人移动，那么具体的攻击点会实时更新，实现追击功能。具体的选点流程如下：

1. 以敌人位姿为中心，3m（自瞄最佳击打距离）为半径，平铺间距为机器人直径的一系列站位点。
2. 将以上 3m 半径圆分割成若干个均匀的扇形部分，每个部分有一个站位点。
3. 计算每个扇形内障碍物点云（来自感知模块）的数量。
4. 排除障碍物点云数量超过一定阈值的扇形对应的站位点，在剩余的站位点中选择距离最近的点作为最终站位点。

下图展示了追击的选点逻辑，坐标轴为机器人当前位置，target 对应点为敌人位姿，蓝色标尺为自瞄最佳打击距离，红色球的位置为候选站位点，红色球的大小为对应扇形部分的障碍物点云数量，红色球的深浅表示点与机器人的距离，越远颜色越深，绿色点为最终选择的站位点。

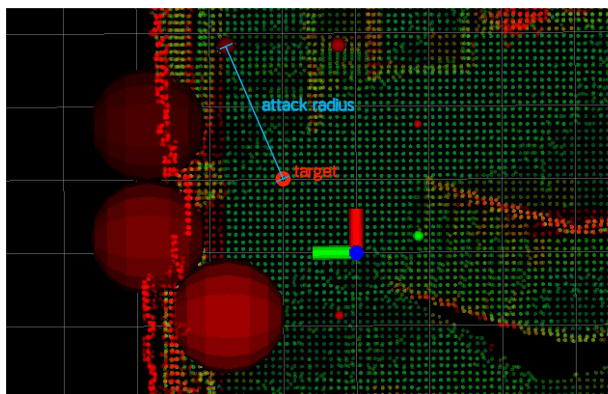


图 17 追击站位点选择算法示意图

导航故障恢复行为

由于撞击或其他原因，机器人可能会到达代价地图的膨胀层中，甚至被膨胀层卡住。Navigaton2 中也设计了相应的对策，即多次尝试旋转和后退，再尝试重新计算代价地图，重新规划路径。这样的方法能够取得一定的脱困效果，但是在多次随机尝试后才有可能找到正确的方向退出，效率比较低。因此，我们重新设计了导航故障恢复行为，使得故障恢复更具有方向性。

当故障恢复行为被触发，机器人以自己为中心，计算一定半径的圆内代价地图数值为 0（即无障碍）的栅格数，若无障碍栅格数量低于阈值，则扩大半径再次统计；若高于阈值，则计算所有圆内无障碍栅格的平均坐标。最后得到这个平均坐标和机器人当前位置的关系，发出命令使得机器人朝向此坐标移动脱困。

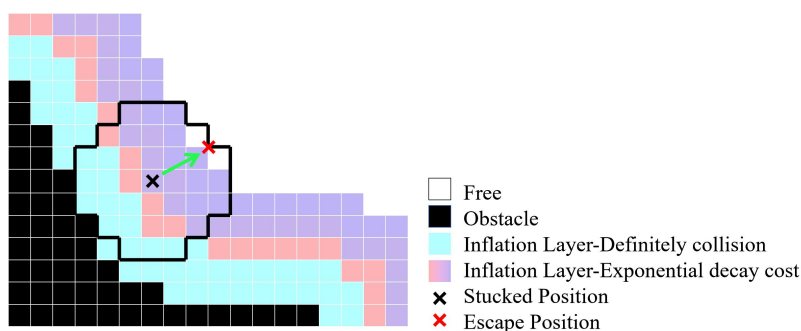


图 18 脱困点计算示意图

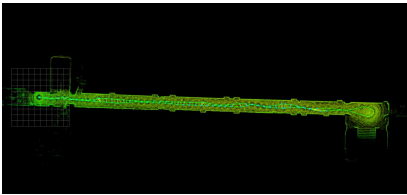
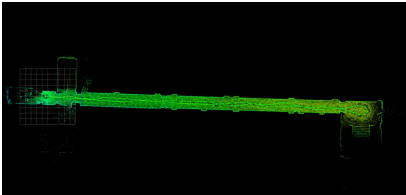
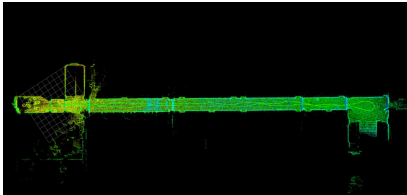
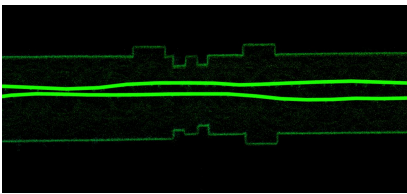
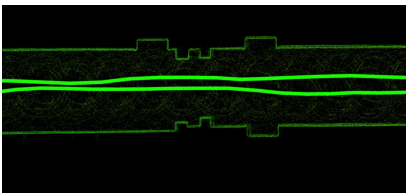
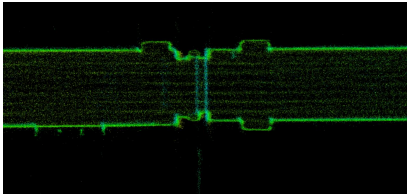
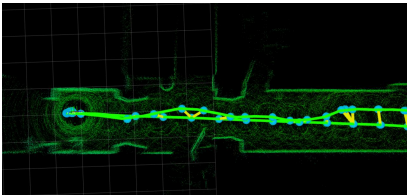
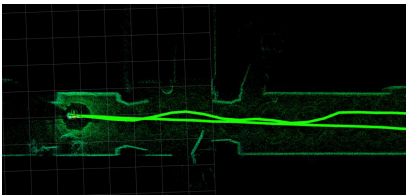
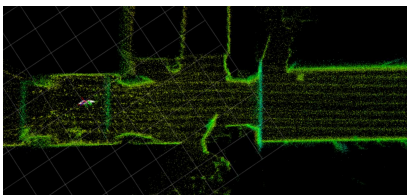
2.1.1.3 算法性能

建图

我们在机器人上使用 MID360 及其内置 IMU 录制了数据集，在离线状态下进行建图对比

算法效果。可以看出，Point-LIO 建图结果中噪声更多，Fast-LIO 和 Fast-LIO-SAM 则具有更高的精度。在经过同一个位置时，由于 Fast-LIO-SAM 有回环的优化（如 Fast-LIO-SAM 视角 3 所示），其取得了比 Fast-LIO 更好的全局一致性，而 Fast-LIO 则出现了一定的“重影”现象。

表 1 建图算法效果对比

	Fast-LIO-SAM	Fast-LIO	Point-LIO
视角 1			
视角 2			
视角 3			

对比三种 SLAM 方法的定位精度，可以看出，三者在 x 和 y 方向上的表现大致相同，但 point-lio 在 z 轴上准确度突出。

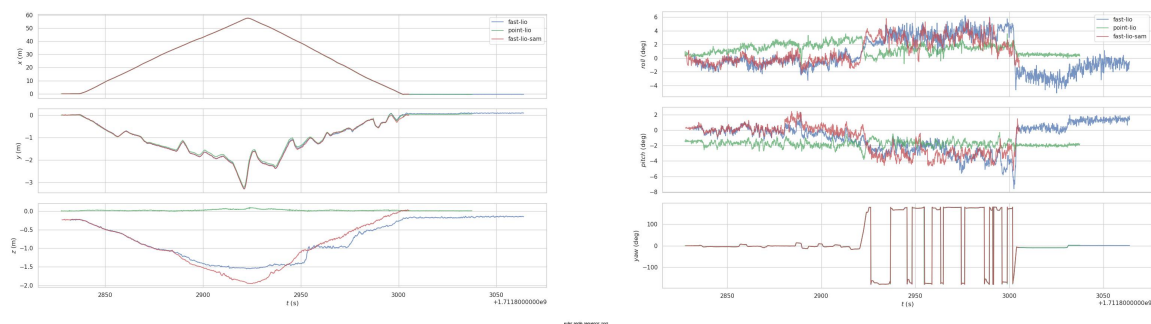


图 19 三种定位方法轨迹对比

定位

在使用 Fast-LIO-SAM 建成的点云地图中运行 Fast-LIO relocalization 模式，以建图时的轨迹作为参考值，分析重定位轨迹的误差结果如图所示。

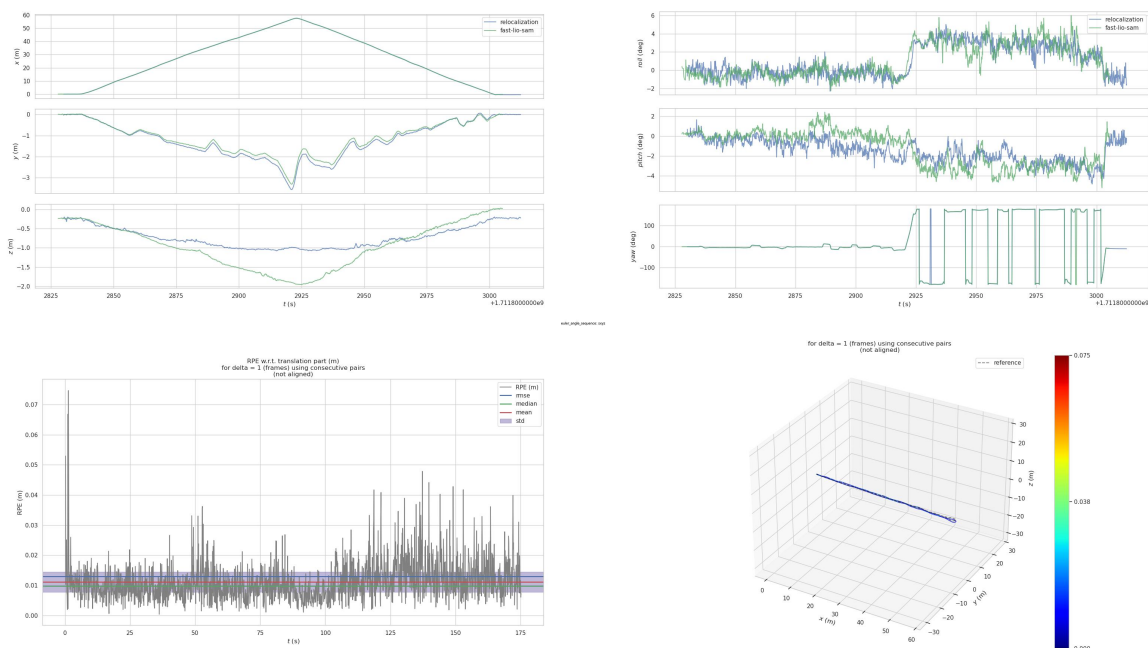


图 20 重定位轨迹和误差分析

可以看出，两者在 x 和 y 方向上的误差极小， rp_y 的角度误差也在 2° 以内。

经过 evo 轨迹分析，重定位轨迹最大误差为 0.074629m，平均误差 0.011192m，标准差 0.006688m，均方根误差 0.013039m。均在可接受的范围内，满足了赛场的定位要求。

感知

Terrain Analysis 模块对于实验室环境的地形分析结果如下图所示。红色部分为不可通行，绿色部分为可通行，可以看出识别的正确率很高，对于低矮的障碍物，如低矮围挡的边界，同样有较好的辨识效果。同时，坡面也被正确分析为可通行路径。

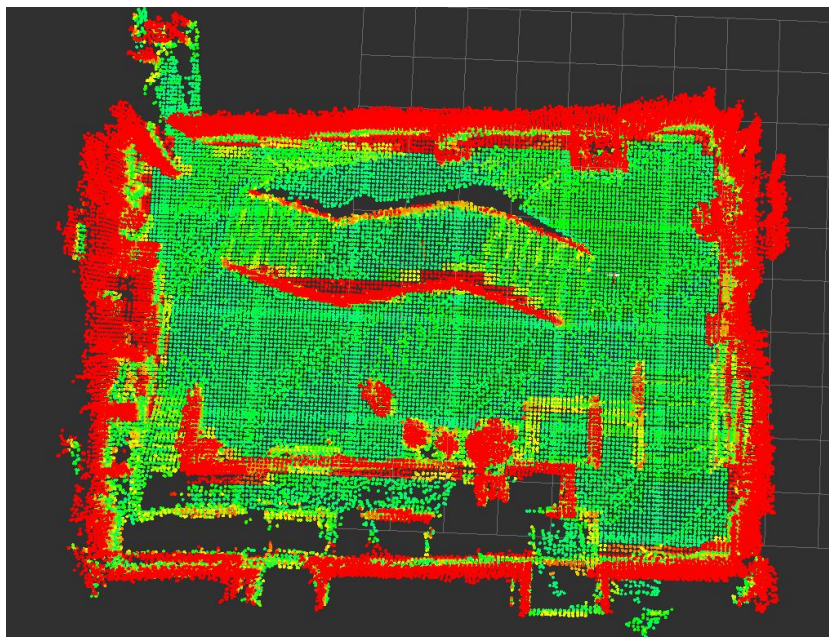


图 21 地形可通过性分析效果

在对于路面法线的估计上，我们在平地、 8° 和 15° 的坡面上进行了精度测试。平地环境下估计坡度为 $0^\circ \pm 1^\circ$ 。 8° 坡上，估计坡度为 $8^\circ \pm 0.8^\circ$ ， 15° 坡上估计坡度为 $15^\circ \pm 1^\circ$ ，坡度估计最大误差 6%。

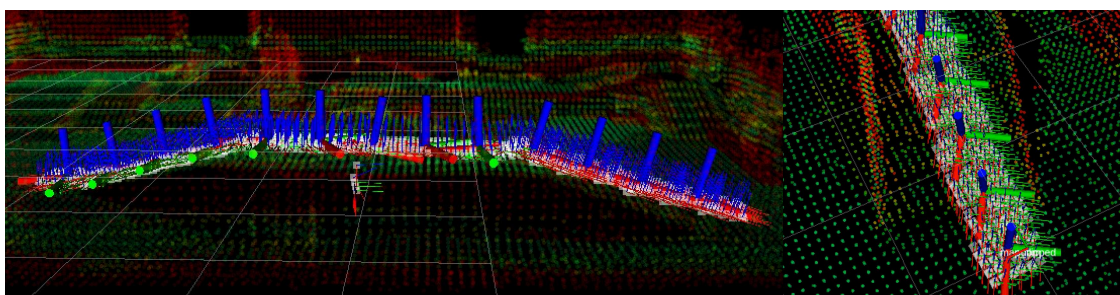


图 22 路径法线分析结果

需要提出的是，以上路面法线的估计测试是在预先较好的地图上进行的。对于预先建好的稠密点云地图，该算法的效果比较好，精度高；但对于实时输入的点云，由于 MID360 点云比较稀疏，算法表现较差。

规划控制

路径规划

为了评判那个路径规划算法更优，我们首先对比了 navigation2 官方给出的五个规划器:Dijkstra、A*、Smacplanner2D、SmacplannerHybrid、Lazy-Theta*。通过对比路径长度和路径质量以及工程调参复杂性等指标综合选择 planner 算法。

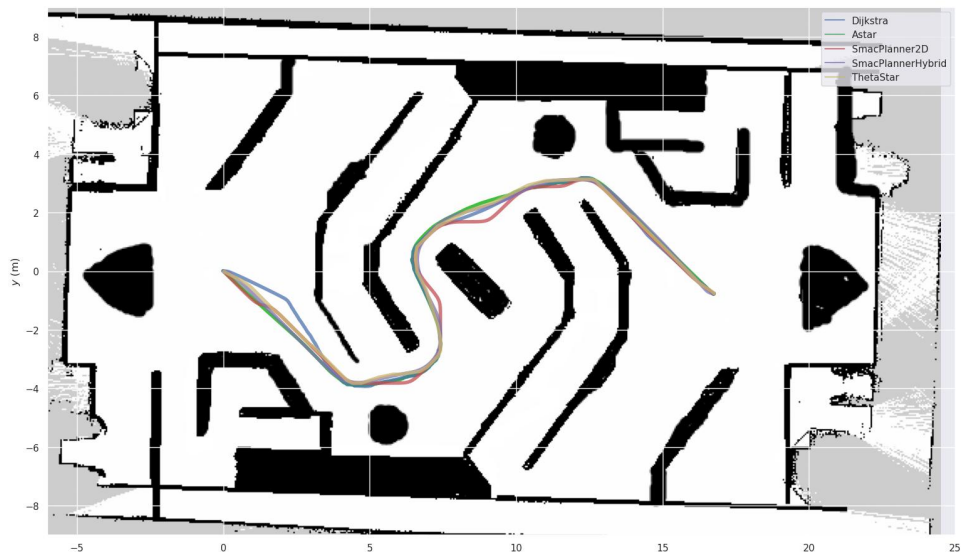


图 23 路径规划算法性能对比

表 2 路径规划算法性能对比

Planner	Path Length
Dijkstra	25.5908
Astar	25.9182
SmacPlanner2D	25.4733
ThetaStar	25.2465

路径平滑

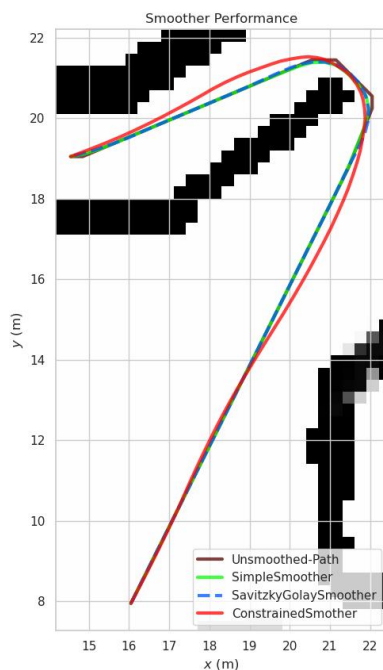


图 24 路径平滑算法性能对比

表 3 路径平滑算法性能对比

Smoother	Path length
None	22.3477
Simple Smoother	22.1171
Savitzky-Golay Smoother	22.1572
ConstrainedSmoo	22.2158

2.2 其它

2.2.1 Git-Flow workflow+Commit Message 规范

本赛季，队内在电控、算法组引入了 Git-Flow 规范进行代码管理，实现了高效的并行开发和版本管理。使得开发按照开发、评审和测试、合并、部署的步骤合理进行。

通过规范 commit message 的格式，仓库迭代更新历程更加清晰，方便进行版本回退，也方便新成员理解某功能研发的原因。

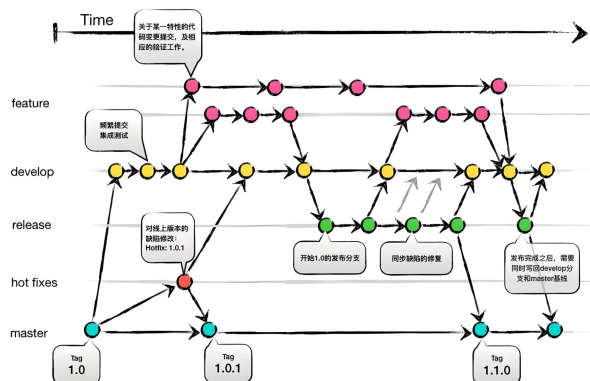


图 25 GitFlow 工作流

2.2.2 持续集成自动构建 Docker 镜像

本赛季，我们为部分代码仓库配置了构建计划。每当 Develop 分支有新的 PR，将自动触发代码测试和镜像构建，完成后，Docker 制品被推送至制品仓库。开发过程中，开发人员可以通过拉取 Docker 镜像获得一致的开发环境，实现环境的一键统一配置；部署过程中，拉取镜像并设置好自启动即可完成部署，更换电脑配置环境也十分迅速，大大提升了代码部署测试的效率。

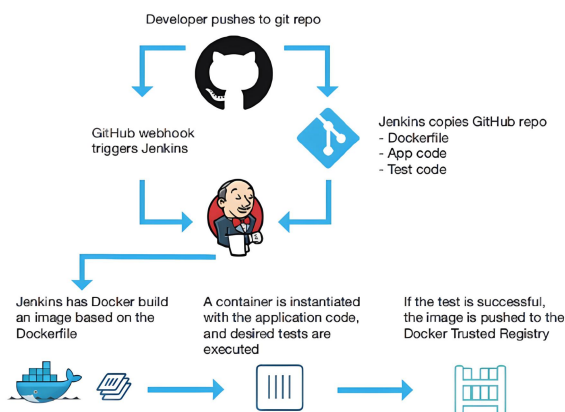


图 26 Docker CI 流程

2.2.3 开发文档记录

本赛季，算法组要求对开发过程进行记录并在例会上进行汇报。记录内容主要围绕开发进度，阶段性成果展示，错误记录。这使得每一个开发者的工作进度有迹可循、为赛季规划，

技术报告提供了宝贵的资料、为新队员留下了参考资料，是本赛季算法组在管理制度上的一次进步。

3. 研发迭代过程

3.1 测试记录

对机器人各项功能点进行测试验收，记录测试环境、测试设备等信息。

测试阶段：算法仿真验证

测试平台：gazebo 仿真平台

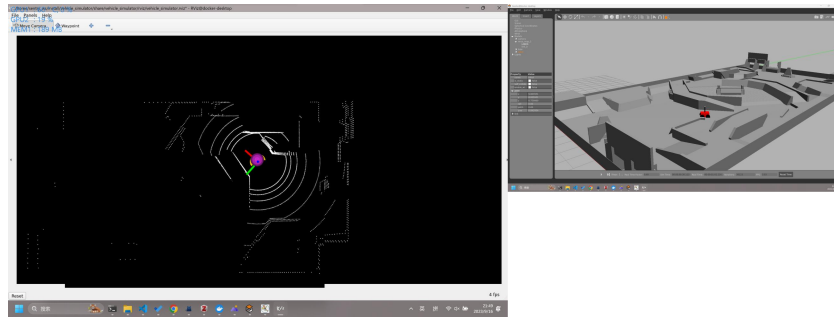


图 27 gazebo 仿真平台

测试内容：初代导航算法测试和方案选择。

测试阶段：算法实际验证

测试平台：迷你麦克纳姆轮小车+MID360+NUC8

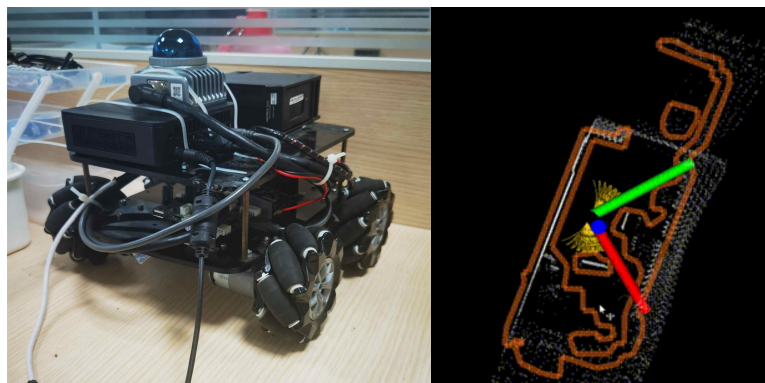


图 28 初代算法验证平台

测试内容：Fast-LIO 建图和定位，DLL 重定位，Far-Planner 全局规划，CMU-local planner 局部控制

测试阶段：算法优化

测试平台：大疆步兵+倒挂 MID360+NUC8



图 29 基于行为树的决策测试

测试内容：实现基于行为树的自主决策、小陀螺导航、追击功能

测试阶段：交流赛

测试平台：初代云台+全向轮底盘+倒挂 MID360+NUC12

测试内容：优化导航算法 (Fast-LIO 建图和定位, DLL 重定位, A*全局规划, CMU-local planner 局部控制)

测试阶段：联盟赛

测试平台：2 代云台+全向轮底盘+倒挂 MID360+NUC11

测试内容：实现高精度建图和精简的定位 (Fast-LIO-SAM 建图, Fast-LIO in prior map 定位和重定位, Theta*全局规划, MPPI 局部控制)

3.2 版本迭代过程记录

迭代过程的时间颗粒度参考兵种、技术点里程碑时间，推荐以月为单位，避免时间跨度太小或过大。

版本号或阶段	功能或性能详细说明	完成时间
	实现基本的建图定位和导航功能 (Fast-LIO 建图和定位, DLL 重定位, Far-Planner 全局规划, CMU-local planner 局部控制)	2023.10.29
	实现基于行为树的自主决策、小陀螺导航、追击功能	2023.11.22
	优化导航算法 (Fast-LIO 建图和定位, DLL 重定位, A*全局规划, CMU-local planner 局部控制)	2024.01.12

版本号或阶段	功能或性能详细说明	完成时间
	实现高精度建图和精简的定位 (Fast-LIO-SAM 建图, Fast-LIO in prior map 定位和重定位, Theta*全局规划, MPPI 局部控制)	2024.3.20

3.3 重点问题解决记录

序号	问题描述	问题产生原因	问题解决方案&实际解决效果	机器人版本号或阶段	解决人员
	FAR-Planner 框架效果不佳，容易规划出不安全的路径，控制器有路径跳变导致电机频繁正反转的问题	由于 MID360 的线数较少，点云比较稀疏，经过 FAR-Planner 的处理后，低矮的障碍物难以被识别。其基于可视图的导航框架相关支持较少，工程化不如 Navigation2 框架，地图难以自定义和修改。	更换了全局规划其和局部控制器，实现了更合理的规划和更平滑的控制。2D 占用地图可以通过图像处理的手段进行编辑和修改。		算法：夏旗
	Fast-LIO2+ICP 框架冗余度高，算力要求高，震动情况下容易失效	机械设计上云台抖动较大，传感器老化 IMU 出现	将 Fast-LIO2 更换为适用于高速移动		算法：夏旗

序号	问题描述	问题产生原因	问题解决方案&实际解决效果	机器人版本号或阶段	解决人员
		漂移, 算法设计不够简洁	机器人的Point-LIO, 用更多的算力换取了较好的鲁棒性。		
	Point-LIO2+ICP 框架冗余度高, 算力要求高, 失效问题还是会出现	算法设计不够简洁, Point-LIO 对参数敏感	研发了 Fast-LIO in prior map 方法, 同时更换了 IMU, 加强了机械结构, 实现了长时间的鲁棒定位		算法: 夏旗 机械: 郑东杰、潘建宇

4. 团队成员贡献

贡献度的标准由团队自行确定，与机器人最终效果和质量整体成正相关即可。

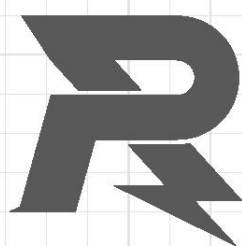
姓名	基本信息 (专业、年级、队内角色)	主要负责工作内容描述	贡献度 (所有成员贡献度合计为 100%)
夏旗	自动化专业，大三，算法-导航组成员	负责哨兵建图、定位、规划控制、决策算法开发和测试	

5. 参考文献

- [1] 【RM2023-技术报告开源】西安电子科技大学 IRobot 战队【RoboMaster 论坛-科技宅天堂】[EB/OL]. [2024-03-31].
<https://bbs.robomaster.com/forum.php?mod=viewthread&tid=22770>.
- [2] 【RM2022-技术报告开源】华南农业大学 Taurus【RoboMaster 论坛-科技宅天堂】[EB/OL]. [2024-03-31]. <https://bbs.robomaster.com/forum.php?mod=viewthread&tid=22172>.
- [3] 【RM2023-技术报告开源】华南农业大学 Taurus 战队【RoboMaster 论坛-科技宅天堂】[EB/OL]. [2024-03-31].
<https://bbs.robomaster.com/forum.php?mod=viewthread&tid=22779>.
- [4] 【RM2023-技术报告开源】太原科技大学 NewMaker 战队【RoboMaster 论坛-科技宅天堂】[EB/OL]. [2024-03-31].
<https://bbs.robomaster.com/forum.php?mod=viewthread&tid=22739&extra=page%3D5&mobile=2>.
- [5] 【RM2023-哨兵技术报告开源】哈尔滨工业大学-技术报告【RoboMaster 论坛-科技宅天堂】[EB/OL]. [2024-03-31].
<https://bbs.robomaster.com/forum.php?mod=viewthread&tid=22790>.
- [6] 【RM2022-技术报告开源】沈阳航空航天大学 TUP 战队【RoboMaster 论坛-科技宅天堂】[EB/OL]. [2024-03-31].
<https://bbs.robomaster.com/forum.php?mod=viewthread&tid=22073>.
- [7] baiyeweiguang/CSU-RM-Sentry: 中南大学 FYT 战队 RM 哨兵机器人上位机算法[EB/OL]. [2024-03-31]. <https://github.com/baiyeweiguang/CSU-RM-Sentry>.
- [8] HUSTLYRM/Hust_Sentry_SLAM: 华中科技大学 ROBOMASTER 2023 赛季 哨兵导航算法开源[EB/OL]. [2024-03-31]. https://github.com/HUSTLYRM/Hust_Sentry_SLAM.
- [9] BAI C, XIAO T, CHEN Y, 等. Faster-LIO: Lightweight Tightly Coupled Lidar-Inertial Odometry Using Parallel Sparse Incremental Voxels[J/OL]. IEEE Robotics and Automation Letters, 2022, 7(2): 4861-4868. DOI:10.1109/LRA.2022.3152830.
- [10] XU W, CAI Y, HE D, 等. FAST-LIO2: Fast Direct LiDAR-Inertial Odometry[J/OL]. IEEE Transactions on Robotics, 2022, 38(4): 2053-2073. DOI:10.1109/TR0.2022.3141876.

- [11] HE D, XU W, CHEN N, 等. Point-LIO: Robust High-Bandwidth Light Detection and Ranging Inertial Odometry[J/OL]. Advanced Intelligent Systems, 2023, 5(7): 2200459. DOI:10.1002/aisy.202200459.
- [12] SHAN T, ENGLT B, MEYERS D, 等. LIO-SAM: Tightly-coupled Lidar Inertial Odometry via Smoothing and Mapping[C/OL]//2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). 2020: 5135-5142. DOI:10.1109/IROS45743.2020.9341176.
- [13] AKAI N. Efficient Solution to 3D-LiDAR-based Monte Carlo Localization with Fusion of Measurement Model Optimization via Importance Sampling[M/OL]. arXiv, 2023[2024-01-17]. <http://arxiv.org/abs/2303.00216>. DOI:10.48550/arXiv.2303.00216.
- [14] Method for registration of 3-D shapes[EB/OL]. [2024-03-29]. <https://www.spiedigitallibrary.org/conference-proceedings-of-spie/1611/1/Method-for-registration-of-3-D-shapes/10.1117/12.57955.short>.
- [15] BIBER P, STRASSER W. The normal distributions transform: a new approach to laser scan matching[C/OL]//Proceedings 2003 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS 2003) (Cat. No.03CH37453): 卷 3. 2003: 2743-2748 卷 3[2024-03-29]. <https://ieeexplore.ieee.org/abstract/document/1249285>. DOI:10.1109/IROS.2003.1249285.
- [16] CABALLERO F, MERINO L. DLL: Direct LIDAR Localization. A map-based localization approach for aerial robots[C/OL]//2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). 2021: 5491-5498. DOI:10.1109/IROS51168.2021.9636501.
- [17] Robust and Precise Vehicle Localization Based on Multi-Sensor Fusion in Diverse City Scenes | IEEE Conference Publication | IEEE Xplore[EB/OL]. [2024-03-27]. <https://ieeexplore.ieee.org/document/8461224>.
- [18] [2003.00368] The Marathon 2: A Navigation System[EB/OL]. [2024-03-31]. <https://arxiv.org/abs/2003.00368>.
- [19] HIMMELSBACH M, HUNDELSHAUSEN F v., WUENSCH H J. Fast segmentation of 3D point clouds for ground vehicles[C/OL]//2010 IEEE Intelligent Vehicles Symposium. 2010: 560-565[2024-03-31]. <https://ieeexplore.ieee.org/abstract/document/5548059>. DOI:10.1109/IVS.2010.5548059.

- [20] ZOU C. baiyeweiguang/linefit_ground_segmentation_ros2[CP/OL].
(2024-03-16) [2024-03-31].
https://github.com/baiyeweiguang/linefit_ground_segmentation_ros2.
- [21] CAO C, ZHU H, YANG F, 等. Autonomous Exploration Development Environment and the Planning Algorithms[C/OL]//2022 International Conference on Robotics and Automation (ICRA). 2022: 8921-8928[2023-09-30].
<https://ieeexplore.ieee.org/abstract/document/9812330>.
DOI:10.1109/ICRA46639.2022.9812330.
- [22] FAST_LIO_SAM/FAST_LIO_SAM at master • kahowang/FAST_LIO_SAM[EB/OL].
[2024-03-31]. https://github.com/kahowang/FAST_LIO_SAM/tree/master/FAST_LIO_SAM.
- [23] RUSU R B. Semantic 3D Object Maps for Everyday Manipulation in Human Living Environments[J/OL]. KI – Künstliche Intelligenz, 2010, 24(4): 345-348.
DOI:10.1007/s13218-010-0059-6.
- [24] Theta*: Any-Angle Path Planning on Grids | Journal of Artificial Intelligence Research[EB/OL]. [2024-03-27].
<https://www.jair.org/index.php/jair/article/view/10676>.
- [25] MACY. Theta*: A Simple Any-Angle Algorithm based on A*[EB/OL]//Movel AI News.
(2020-08-31) [2024-05-13]. <https://news.movel.ai/theta-star>.
- [26] WILLIAMS G, DREWS P, GOLDFAIN B, 等. Aggressive driving with model predictive path integral control[C/OL]//2016 IEEE International Conference on Robotics and Automation (ICRA). 2016: 1433-1440[2024-01-14].
<https://ieeexplore.ieee.org/document/7487277>. DOI:10.1109/ICRA.2016.7487277.
- [27] Matlab 用采样的离散点做前向模拟三次样条生成路径点-CSDN 博客[EB/OL].
[2023-10-04]. https://blog.csdn.net/qq_39266065/article/details/119734986.



邮箱: robomaster@dji.com

论坛: <http://bbs.robomaster.com>

官网: <http://www.robomaster.com>

电话: 0755-36383255 (周一至周五10:30-19:30)

地址: 广东省深圳市南山区西丽街道仙茶路与兴科路交叉口大疆天空之城T2 22F